The two functions, `formatAbs` and `formatFactorial`, are almost identical. If we like, we can generalize these to a single function, `formatResult`, which accepts as an argument the *function* to apply to its argument:

```
def formatResult(name: String, n: Int, f: Int => Int) = {
  val msg = "The %s of %d is %d."
  msg.format(name, n, f(n))
}
```

`f` is required to be a function from `Int` to `Int`.

Our `formatResult` function is a higher-order function (HOF) that takes another function, called `f` (see sidebar on variable-naming conventions). We give a type to `f`, as we would for any other parameter. Its type is `Int => Int` (pronounced "int to int" or "int arrow int"), which indicates that `f` expects an integer argument and will also return an integer.

Our function `abs` from before matches that type. It accepts an `Int` and returns an `Int`. And likewise, `factorial` accepts an `Int` and returns an `Int`, which also matches the `Int => Int` type. We can therefore pass `abs` or `factorial` as the `f` argument to `formatResult`:

```
scala> formatResult("absolute value", -42, abs)
res0: String = "The absolute value of -42 is 42."

scala> formatResult("factorial", 7, factorial)
res1: String = "The factorial of 7 is 5040."
```

Variable-naming conventions

It's a common convention to use names like `f`, `g`, and `h` for parameters to a higher-order function. In functional programming, we tend to use very short variable names, even one-letter names. This is usually because HOFs are so general that they have no opinion on what the argument should actually *do*. All they know about the argument is its type. Many functional programmers feel that short names make code easier to read, since it makes the structure of the code easier to see at a glance.

2.5 Polymorphic functions: abstracting over types

So far we've defined only *monomorphic* functions, or functions that operate on only one type of data. For example, `abs` and `factorial` are specific to arguments of type `Int`, and the higher-order function `formatResult` is also fixed to operate on functions that take arguments of type `Int`. Often, and especially when writing HOFs, we want to write code that works for *any* type it's given. These are called *polymorphic functions*,[5] and in the chapters ahead, you'll get plenty of experience writing such functions. Here we'll just introduce the idea.

[5] We're using the term *polymorphism* in a slightly different way than you might be used to if you're familiar with object-oriented programming, where that term usually connotes some form of subtyping or inheritance relationship. There are no interfaces or subtyping here in this example. The kind of polymorphism we're using here is sometimes called *parametric polymorphism*.